

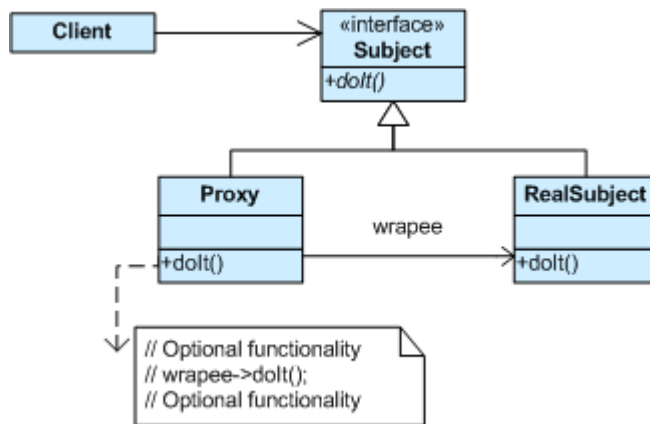
Proxy Design Pattern

Intent

- Provide a surrogate or placeholder for another object to control access to it.
- Use an extra level of indirection to support distributed, controlled, or intelligent access.
- Add a wrapper and delegation to protect the real component from undue complexity.

Structure

By defining a Subject interface, the presence of the Proxy object standing in place of the RealSubject is transparent to the client.



Example

The Proxy provides a surrogate or place holder to provide access to an object. A check or bank draft is a proxy for funds in an account. A check can be used in place of cash for making purchases and ultimately controls access to cash in the issuer's account.

```
import java.io.*; import java.net.*;

// 5. To support plug-compatibility between
// the wrapper and the target, create an interface
interface SocketInterface {
    String readLine();
    void writeLine( String str );
    void dispose();
}

public class ProxyDemo {
    public static void main( String[] args ) {

        // 3. The client deals with the wrapper
```

```

SocketInterface socket = new SocketProxy( "127.0.0.1", 8189,
    args[0].equals("first") ? true : false );

String str = null;
boolean skip = true;
while (true) {
    if (args[0].equals("second") && skip) {
        skip = ! skip;
    }
    else {
        str = socket.readLine();
        System.out.println( "Receive - " + str ); // java ProxyDemo first
        if (str.equals("quit")) break; // Receive - 123 456
    } // Send ---- 234 567
    System.out.print( "Send ---- " ); // Receive - 345 678
    str = Read.aString(); //
    socket.writeLine( str ); // java ProxyDemo second
    if (str.equals("quit")) break; // Send ---- 123 456
    } // Receive - 234 567
    socket.dispose(); // Send ---- 345 678
}
}

class SocketProxy implements SocketInterface {
    // 1. Create a "wrapper" for a remote,
    // or expensive, or sensitive target
    private Socket socket;
    private BufferedReader in;
    private PrintWriter out;

    public SocketProxy( String host, int port, boolean wait ) {
        try {
            if (wait) {
                // 2. Encapsulate the complexity/overhead of the target in the wrapper
                ServerSocket server = new ServerSocket( port );
                socket = server.accept();
            } else
                socket = new Socket( host, port );
            in = new BufferedReader( new InputStreamReader(
                socket.getInputStream() ));
            out = new PrintWriter( socket.getOutputStream(), true );
        } catch( IOException e ) {
            e.printStackTrace();
        }
    }

    public String readLine() {
        String str = null;
        try {
            str = in.readLine();
        } catch( IOException e ) {
            e.printStackTrace();
        }
        return str;
    }

    public void writeLine( String str ) {
        // 4. The wrapper delegates to the target
        out.println( str );
    }

    public void dispose() {
        try {
            socket.close();
        } catch( IOException e ) {
            e.printStackTrace();
        }
    }
}

```